

Research Assignment: HTTP State Preservation & Database Migrations

Question 1: State Preservation in HTTP

The Hypertext Transfer Protocol (HTTP) is inherently stateless, meaning that each request-response cycle is treated as an independent transaction without any built-in memory of previous interactions. To build dynamic web applications that require user continuity—such as shopping carts, user profiles, or sticky note management—developers rely on three primary mechanisms to bridge this stateless gap: **cookies**, **server-side sessions**, and **authentication**.

- **Cookies:** Small pieces of data stored directly on the client's browser by the server via HTTP response headers (**Set-Cookie**). The browser automatically attaches these cookies to subsequent requests using the **Cookie** header, allowing the server to recognize returning clients. Because client-side cookies can be inspected or manipulated, they are rarely used to store sensitive data directly; instead, they typically hold a secure, opaque session identifier.
- **Sessions:** Moves the actual state data to the server side (such as a database or cache). When a user logs in or interacts with the site, the server creates a unique session record mapped to that user and hands the browser a session ID cookie. Whenever a request is made, the server looks up the session ID to retrieve user-specific data instantly.
- **Authentication:** Verifies the identity of the user (e.g., token-based authentication using JSON Web Tokens or Django's built-in session authentication). Once authenticated, the server binds the authorization status to the session or token, ensuring subsequent requests are secure, authorized, and aware of the user's operational context.

Question 2: Database Migrations to MariaDB

Moving a Django web application from a lightweight development database like SQLite to an enterprise-grade production relational database management system (RDBMS) like MariaDB requires a structured configuration and migration procedure.

The process involves the following key steps:

1. **Environment Preparation:** Install the necessary database driver package (such as `mariadb` or `mysqlclient`) in your Python virtual environment so Django can communicate with the MariaDB server.
2. **Configuration Update:** Update the project configuration in `settings.py` within the `DATABASES` dictionary. Replace the default SQLite file engine with MariaDB parameters:
 - Set the engine to `django.db.backends.mysql`
 - Specify the database name, user credentials, host, and port.
3. **Schema Deployment:** Deploy the database schema. Because Django's migration files are database-agnostic, running `python manage.py migrate` automatically translates and executes the schema creation statements (tables, foreign keys, and indexes) directly onto the MariaDB instance.
4. **Data Preservation (Optional/As Needed):** If existing data needs to be preserved from SQLite during the transition, utilize Django's data serialization utilities (`python manage.py dumpdata > data.json`) before switching the configuration, followed by loading the serialized data into the new database using `python manage.py loaddata data.json` after running the initial migrations.
5. **Testing:** Perform rigorous testing to ensure all constraints, character encodings (typically `utf8mb4` for MariaDB), and queries function identically in the production environment.